

信息安全基础综合设计实验

Lecture 02

李经纬

电子科技大学

课程回顾

Linux简单编程

➤ 命令行环境

- echo, PATH, ls, mv, cp, mkdir, man, < >

➤ 编译指令：`g++ -o <可执行文件名> <源文件名>`

➤ Makefile

- 由一系列**规则**构成
- 每条规则包含**目标、依赖、指令**

```
TARGET: PREREQUISITE ...  
        COMMAND ...  
        COMMAND ...
```

Shell 脚本基础

Shell脚本

➤一种解释型脚本语言

- 直接由 Shell 解释器执行，快速、灵活、无需编译

➤用途：自动化、系统管理、任务调度

- 轻松地将不同的程序、工具和服务连接起来，构建成一个完整的工作流
- 天生用来与操作系统和文件系统交互

➤**Bash**是为 GNU 计划编写的 Unix shell，是 Linux 和 macOS 下的默认 shell

- 其他Shell包括Zsh、Fish

脚本结构

➤脚本第一行告知系统如何执行该脚本

```
#!/bin/bash
```

```
# 表示注释
```

```
# 后续输入程序命令
```

变量与字符串

➤ 变量赋值与访问

- 在 bash 中为变量赋值的语法是 `foo=bar`，访问语法为 `$foo`
- **`foo = bar`（使用空格隔开）是不能正确工作的**

➤ 字符串处理

- **单引号（'）**：原义字符串，**不进行**变量替换
- **双引号（"）**：进行变量替换

```
$ foo=bar          # 赋值（注意：等号两边不能有空格）
$ echo "$foo"      # 输出：bar
$ echo '$foo'      # 输出：$foo
```

条件判断

➤ 返回码规则

- 0表示true，任何非零值（1-255）都表示false

➤ 逻辑操作符

- `&&` 和 `||` 属于短路运算符

```
echo "Always executed" || echo "Only executed if first command fails"  
echo "Always executed" && echo "Only executed if first command does NOT fail"
```

- 同一行多个命令用 `;` 分隔（无论左边成功还是失败，右边都会执行）

条件控制流

- If条件控制语句的三种形式
- [条件]是**测试表达式**，判断某个条件是否为真，并返回一个退出状态码（0 代表真，非 0 代表假）
 - **if**和**while**等控制流语句的核心

```
if [ 条件 ]; then
    # 如果条件为真，执行这里的命令
fi
if [ 条件 ]; then
    # 如果条件为真，执行这里的命令
else
    # 否则，执行这里的命令
fi
if [ 条件1 ]; then
    # 如果条件1为真
elif [ 条件2 ]; then
    # 否则，如果条件2为真
elif [ 条件3 ]; then
    # 否则，如果条件3为真
else
    # 所有条件都为假
fi
```

测试表达式 I

➤方括号 [条件表达式]

- 每个方括号和其中的每个元素之间都必须用空格隔开
- 实际上是调用了一个名为 `test` 的命令

操作符	描述	示例
<code>-z</code>	字符串为空	<code>[-z "\$my_var"]</code>
<code>-n</code>	字符串不为空	<code>[-n "\$my_var"]</code>
<code>=</code> / <code>==</code>	字符串相等	<code>["\$str1" = "\$str2"]</code>
<code>!=</code>	字符串不相等	<code>["\$str1" != "\$str2"]</code>
<code>-eq</code>	整数等于	<code>["\$num1" -eq "\$num2"]</code>
<code>-ne</code>	整数不等于	<code>["\$num1" -ne "\$num2"]</code>
<code>-gt</code>	整数大于	<code>["\$num1" -gt "\$num2"]</code>
<code>-lt</code>	整数小于	<code>["\$num1" -lt "\$num2"]</code>
<code>-f</code>	文件存在且是普通文件	<code>[-f "/path/to/file.txt"]</code>
<code>-d</code>	目录存在且是目录	<code>[-d "/path/to/dir"]</code>
<code>-e</code>	文件或目录存在	<code>[-e "/path/to/something"]</code>

变量！ 最好用双引号 `$var` 包裹变量，以避免在变量为空或包含空格时出错

测试表达式 II

➤ 双重方括号 `[[ 条件表达式 ]]`

- 现代Shell的扩展，推荐的写法

➤ 主要优势

- 变量即使不加双引号也能正确处理空格
- 可以直接使用 `&&`（逻辑与）和 `||`（逻辑或）
- 可以使用 `==` 进行字符串通配符匹配

```
# 检查变量是否以 "test" 开头
if [[ "$my_var" == test* ]];
then
    echo "变量以 test 开头"
fi
```

```
# 检查文件是否存在且可写
if [[ -e "/path/to/file" &&
-w "/path/to/file" ]]; then
    echo "文件存在且可写"
fi
```

测试表达式 III

➤ 双重括号 ((条件表达式))

- 专门用于算术运算的测试表达式
- 允许像在 C 语言中一样进行数学比较和赋值

```
num1=10
num2=20
if (( num1 < num2 )); then
    echo "num1 小于 num2"
fi
# 也可以进行复杂的算术运算
if (( (num1 + num2) * 2 > 50 )); then
    echo "结果大于 50"
fi
```

通常建议优先使用 [[]] 来进行字符串和文件测试，使用 (()) 来进行算术比较

循环控制流

- **for** 循环用于遍历一个列表或数字递增/递减（C语言风格）
- **while** 循环只要条件为真，就会一直执行

```
# 遍历列表
for fruit in apple banana orange; do
    echo "我喜欢吃 $fruit"
done
# 数字递增
for (( i=1; i<=3; i++ )); do
    echo "这是第 $i 次循环"
done
```

```
count=1

while [ $count -le 3 ]
do
    # 只要条件为真，就执行这里的代码
    echo "当前计数是: $count"
    count=$(( count + 1 ))
done
```

函数定义

➤ 函数语法：使用 `function` 关键字

```
function function_name {  
    # 函数体：要执行的命令  
    echo "这是我的第一个 Shell 函数！"  
}  
  
# 调用函数  
function_name
```

输出：这是我的第一个 Shell 函数！

函数传参

➤函数传参：参数在函数内部被称为位置参数

- `$1`、`$2`、`$3`...：分别代表第一个、第二个、第三个参数
- `$#`：参数的总个数。
- `$*` 或 `$@`：所有参数

```
# 定义一个接受参数的函数
function greet_user {
    echo "你好，$1！你今年 $2 岁了。"
}
# 调用函数并传递参数
greet_user Alice 25
```

输出：你好，Alice！你今年 25 岁了。

➤当位置参数在函数外时，代表脚本的传入参数，`$0`代表脚本名称

函数返回值 I

➤使用退出状态码来返回值

- 0：表示函数成功执行
- 非零值：表示函数执行失败
- 可以在函数体中用 **return** 命令来指定返回值，然后在函数调用后，通过 **\$?** 变量来获取这个值

```
# 定义一个有返回值的函数
function check_file {
    if [ -f "$1" ]; then
        return 0    # 文件存在，返回成功
    else
        return 1    # 文件不存在，返回失败
    fi
}

# 调用函数并检查返回值
check_file "/path/to/my_file.txt"

if [ $? -eq 0 ]; then
    echo "文件存在。"
else
    echo "文件不存在。"
fi
```


函数返回值 II

- 如果需要返回字符串或其他数据，通常将它打印到标准输出，然后用命令替换（例如 `var=$(my_function)`）来捕获

```
function get_timestamp {  
    echo "$(date +%Y-%m-%d) "  
}  
  
# 捕获函数输出到变量中  
current_date=$(get_timestamp)  
echo "当前日期是：$current_date"
```

Shell 文本处理工具

find命令 I

➤ `find` [搜索路径] [搜索条件] [对结果执行的操作]

➤ 搜索路径

- 当前目录： `.`
- 上级目录： `..`
- 根目录： `/`
- 主目录： `~`

find命令 II

- `find` [搜索路径] [搜索条件] [对结果执行的操作]
- 搜索条件（最强大的部分，可以组合各种条件精确地筛选文件）
 - `-name`：按文件名精确查找，支持通配符（`*`, `?`）
 - `-iname`不区分大小写
 - `-type`：根据文件类型查找
 - `f` 普通文件，`d` 目录，`l` 软链
 - `-size`：根据文件大小查找
 - `b` 字节，`c` 字符，`k` 千字节，`M` 兆字节，`G` 吉字节
 - `-mtime`：按文件修改时间查找，单位是天
 - `+n` `n` 天前修改的，`-n` `n` 天内修改的，`n` 正好是`n`天前修改的

find命令 III

➤ `find` [搜索路径] [搜索条件] [对结果执行的操作]

➤ 对结果执行的操作

- `-print` : 打印找到的文件路径
- `-exec` : 对每个找到的文件执行指定的命令, 语法为 `-exec command {} \;` 或 `-exec command {} +`
 - `{}` 是一个占位符, 代表 `find` 找到的当前文件路径
 - `\;` 结尾表示对每个找到的文件都执行一次命令
 - `+` 结尾表示将所有找到的文件作为参数一次性传递给命令

```
find /home/myuser/documents -type f -mtime +30 -size +50M -exec ls -l {} \;
```

grep命令 I

➤查找匹配模式的行：`grep [选项] '模式' [文件名]`

选项	描述	示例
<code>-i</code>	忽略大小写	<code>grep -i 'error' log.txt</code>
<code>-v</code>	反向查找，显示不匹配模式的行	<code>grep -v 'success' log.txt</code>
<code>-r</code>	递归查找，在指定目录及其子目录中查找。	<code>grep -r 'TODO' .</code>
<code>-n</code>	显示行号	<code>grep -n 'main' my_code.c</code>
<code>-l</code>	仅显示文件名，只列出包含匹配行的文件名	<code>grep -l 'password' *.conf</code>
<code>-c</code>	计数，只显示匹配的行数	<code>grep -c 'Failed' my_app.log</code>
<code>-w</code>	全词匹配，只匹配完整的单词，而不是子字符串	<code>grep -w 'log' file.txt</code>
<code>-E</code>	扩展正则表达式，允许使用更复杂的正则表达式	<code>grep -E '^(a' file.txt</code>

grep命令 II

➤查找匹配模式的行：`grep [选项] '模式' [文件名]`

在 app.log 文件中查找所有包含 "error" 的行，并显示行号

```
grep -n 'error' app.log
```

先用 `ps` 列出所有进程，然后用 `grep` 找出其中包含 "nginx" 的进程

```
ps aux | grep nginx
```

在当前目录及其所有子目录中查找所有 .py 文件，找出包含 "import os" 的行。

```
grep -r 'import os' . --include='*.py'
```

查找以 "START" 开头并以 "END" 结尾的行

```
grep '^START.*END$' my_file.txt
```

查找包含 "cat" 或 "dog" 的行

```
grep -E 'cat|dog' my_pets.txt
```

grep命令 II

➤查找匹配模式的行：`grep [选项] '模式' [文件名]`

在 app.log 文件中查找所有包含 "error" 的行，并显示行号

```
grep -n 'error' app.log
```

先用 `ps` 列出所有进程，然后用 `grep` 找出其中包含 "nginx" 的进程

```
ps aux | grep nginx
```

在当前目录及其所有子目录中查找所有 .py 文件，找出包含 "import os" 的行。

```
grep -r 'import os' . --include='*.py'
```

查找以 "START" 开头并以 "END" 结尾的行

```
grep '^START.*END$' my_file.txt
```

查找包含 "cat" 或 "dog" 的行

```
grep -E 'cat|dog' my_pets.txt
```

正则表达式	描述
.	匹配单个字符
*	匹配前一个字符零次或多次
^	匹配行首
\$	匹配行尾
[]	匹配方括号中的任意一个字符

awk命令 I

➤按列（字段）处理数据：`awk '模式 { 操作 }' 文件名`

- **模式**：可选，一个正则表达式或一个条件筛选出想要处理的行
- **操作**：必需，一个或多个命令，用来处理匹配到的行
- **文件名**：如果不指定文件名，从标准输入（比如管道|）读取

➤工作原理

- **逐行读取**：从输入文件或标准输入中读取一行文本
- **字段分割**：自动将这行文本分割成不同的**字段**，默认分割符是空格或Tab
- **模式匹配**：将这行文本与指定的**模式**进行比较
- **执行操作**：如果该行匹配模式，或者没有指定模式，awk 就会执行定义好的**操作**
- **循环**：重复上述步骤，直到处理完所有行

awk命令 II

➤按列（字段）处理数据：`awk '模式 { 操作 }' 文件名`

➤内置变量

- **\$0**：代表**整个**当前行
- **\$1、\$2、\$3...**：代表当前行的**第一个、第二个、第三个...字段**
- **NF**：当前行**字段（列）的总数**
- **NR**：当前**行号**

awk命令 III

```
Alice 30 IT  
Bob 25 Marketing  
Charlie 45 Finance  
David 25 IT
```

已知数据文件employees.txt

打印每一行的第一个和第三个字段

```
awk '{ print $1, $3 }' employees.txt
```

只匹配年龄小于 30 的行

```
awk '$2 < 30 { print $1, $2 }' employees.txt
```

计算平均年龄

```
awk 'BEGIN { total_age=0 } { total_age+=$2 } END { print "平均年龄 :",  
total_age/NR }' employees.txt
```

awk命令 III

```
Alice 30 IT
Bob 25 Marketing
Charlie 45 Finance
David 25 IT
```

已知数据文件employees.txt

打印每一行的第一个和第三个字段

```
awk '{ print $1, $3 }' employees.txt
```

只匹配年龄小于 30 的行

```
awk '$2 < 30 { print $1, $2 }' employees.txt
```

计算平均年龄

```
awk 'BEGIN { total_age=0 } { total_age+=$2 } END { print "平均年龄 :",
total_age/NR }' employees.txt
```

BEGIN和END特殊模式

- **BEGIN**块在开始处理文件之前执行一次
- **END**块在处理完所有行之后执行一次

自学资源

- Shell脚本 : <https://learnxinyminutes.com/zh-cn/bash/>
- Awk : <https://learnxinyminutes.com/zh-cn/awk/>

作业

作业要求

➤作业内容：日志文件分析

➤评分等级：

- **课堂上**完成作业，正确解释代码，通过助教登记，获得平时成绩加分
- 课后按时（本周日24点之前）完成作业并发送至邮箱**zihasyu@proton.me**，获得本次作业平时成绩
 - 邮件题目：【周X】姓名+学号
- 未完成作业，无法获得本次作业平时成绩